RTL and Verification Coding Standards

1. Purpose

These guidelines define common rules for writing, organizing, documenting, and optimizing RTL and verification code.

The goal is to ensure code is:

- Readable and consistent.
- Reusable and maintainable.
- Synthesizable where required.
- Easy to verify and debug.
- Efficient in hardware implementation.

Rules marked **MUST** are mandatory. Rules marked **SHOULD** are recommended unless there is a valid reason to deviate.

2. Naming conventions

Use descriptive names that communicate the purpose of each element.

- Modules: lowercase with underscores, e.g. "fifo_controller".
- Files: match the main module name, e.g. "fifo_controller.sv".
- Inputs: suffix "_i".
- Outputs: suffix "_o".
- Bidirectional signals: suffix "_io".
- Internal registered signals: suffix "_q".
- Next-state signals: suffix "_d".
- Parameters and local parameters: uppercase, e.g. "DATA_WIDTH".
- Enumerated types: descriptive names ending in "_t".
- Tasks and functions: descriptive lowercase names.
- Test names: describe the functionality being tested.

Use consistent naming throughout the project. Avoid unnecessary abbreviations and generic names such as "temp", "data1", or "signal".

3. RTL structure

Organize modules consistently:

1. Module declaration and parameters.

2. Port declarations.

3. Internal signals and registers.

4. Combinational logic.

5. Sequential logic.

6. Output assignments.

7. Generate blocks, if required.

Keep modules focused on one main responsibility. Separate independent functionality into smaller modules when this improves readability or reuse.

Avoid unnecessarily large modules and deeply nested logic.

4. Parameterization and reuse

Modules should be parameterized when their functionality can reasonably be reused with different configurations.

- Define parameters at module level.
- Use parameters for widths, depths, and configurable operating modes.
- Define valid parameter ranges.
- Check parameter compatibility where necessary.
- Avoid hardcoded values when they represent configurable design characteristics.

Parameterization should not make the design unnecessarily complicated.

5. Hardware optimization

Prioritize correct and readable RTL before optimization.

Consider the following:

- Area: avoid unnecessary registers and duplicated logic.
- Timing: avoid unnecessarily deep combinational paths.
- Power: avoid unnecessary switching activity.
- Memory: use coding styles that support the intended memory inference.
- Arithmetic: consider resource sharing and appropriate operator selection.
- Clocking: avoid unnecessary generated clocks and use clock enables where appropriate.

Use synthesis reports, timing analysis, and resource utilization reports to evaluate optimization decisions. Make changes based on measurable results.

6. Comments and documentation

Add comments when:

- Logic is complex or non-obvious.
- An implementation decision needs explanation.
- A workaround or limitation exists.

- Timing or protocol behavior requires clarification.

Document module interfaces, parameters, assumptions, and important design constraints.

Remove outdated, redundant, or misleading comments.

7. Verification code standards

Verification code should follow the same naming and readability rules as RTL.

- Keep stimulus generation separate from checking.
- Use independent reference models.
- Use scoreboards when transaction or data comparison is required.
- Keep drivers responsible for driving interfaces.
- Keep monitors responsible for observing interface activity.
- Use assertions for temporal behavior and design invariants.
- Use functional coverage to measure verification scenarios.
- Make tests self-checking whenever possible.
- Avoid duplicating verification logic across tests.

Use UVM when the complexity and reuse requirements justify it. Do not introduce unnecessary verification infrastructure for simple components.

8. Formatting and readability

- Use consistent indentation.
- Keep one statement per line.
- Use meaningful whitespace to separate logical sections.
- Avoid excessively long lines.
- Group related declarations and logic together.
- Keep procedural blocks reasonably small.
- Avoid unnecessary nesting.
- Follow the same formatting conventions across the project.

9. Code quality and review

Before considering code complete, check that:

- Naming conventions are followed.
- The module has a clear responsibility.
- Signal widths and signedness are correct.
- Sequential and combinational logic are properly separated.
- No unintended latches or multiple drivers exist.
- Parameters and supported configurations are defined.
- Comments and documentation are accurate.
- RTL passes lint and compilation.

- Verification results support the intended behavior.
- Synthesis results are consistent with design expectations.